Fix: One card per service on Services

Type: Fix

Size: light - the same rearrangement People got on 2026-10-25, on the Services page: a service's hours move into its card and save with its one Save; the saves and their rules already exist and do not change.

Status: verified

Branch: fix/one-card-per-service-on-services

The problem

Since feature 12k, each service on Settings, Services shows two boxes: its row (name, length, Live or Hidden, Change) and, under it, a second row with its name again and "Runs whenever the business is open", which opens its hours with a Save of its own. It is the shape People had until the one-card fix merged this morning. Walking Riverbend Clinic (dev) on 2026-10-25, Frank read it the same way ("cabinet consultation 45 minutes, change, like what the heck is that? And then there is another runs whenever the business is open"), and asked for one box, as on People (his yes, 2026-10-25).

- `frontend/components/settings/services-screen.tsx` (`hoursCardOf`, the row and the open form)
- `frontend/components/settings/service-form.tsx`, `service-hours-card.tsx`, `own-hours-card.tsx`
- `frontend/components/settings/person-hours-part.tsx` (the part People's card already uses)

The fix

One box per service, closed or open, the way People works now:

- **The closed row** says it all in one place: name, length, Live or Hidden, when it runs ("Runs whenever the business is open", or "Runs on its own week · Mon, Tue", and any one-off dates), and the button. No second row under it.
- **Change opens the whole card**: name, length, live or hidden, who does it and where, and "When it runs" (whenever the business is open, or its own week, and its one-off dates). **One Save** saves all of it; **Cancel** drops what was not saved. Nothing has a Save of its own.
- **One Save, in order**: first the hours part checks its own form (a window that ends before it starts is caught here, before anything is saved), then the service itself (name, length, live), who does it, and its hours, each through its existing route. A refusal at any of them keeps the card open on its error, the parts before it stay saved (as now), and Cancel then says what was saved and lists what those saves left outside. Hours change only when edited. The card holds its own parts for the whole Save, so opening another service mid-Save cannot mix two cards (the lesson of F-511).

- **After the save**, the page says what happened in one notice and lists the upcoming bookings to call about: the ones a new length no longer fits and the ones new hours leave outside, each under its own title, until the next save.
- **A new service** gets no hours part: it runs whenever the business is open, and Change gives it its own hours afterwards (as a new person on People).
- **A member** (who may not change the business) gets "See" instead of "Change": the same card, every field read-only, no Save, "Close" instead of Cancel, so members still see when each service runs. The "Your role cannot change the services" line stays above the list.
- **One hours part for both pages**: People's hours part becomes the one both cards use, in each page's words, so the two never drift; the separate hours cards (`own-hours-card.tsx` , `service-hours-card.tsx`) have no reader left and go.
- New words in English and Spanish only where a page needs one it lacks ("See", "Close", the outside-its-hours title on the page).

It must not break: every save's own checks and refusals (a name taken, a length out of range, nobody to do it while live, an hours window ending before it starts), the list of bookings a new length no longer fits, People's card and its hours, and the Spanish screens.

Build steps

- [x] 1. **One card per service.** The hours part joins the service's card and its one Save; the second row goes; the closed row carries the hours line; members get the card read-only. **Done when:** on http://localhost:3400/settings/services (Riverbend Clinic (dev), then Summit Painting (dev)) every service shows one box; changing Body Wrap's length and its own week in one open card and pressing Save saves both (read back after a reload), and Cancel drops both; an hours error (a window ending before it starts) keeps the card open on it and saves nothing; a booking a shorter week leaves outside is listed on the page; People's card still saves its hours; every new word reads in Spanish; builds, lint and every suite pass.

Verify

Open Settings, Services. Each service is one box that says its length and when it runs. Press Change on one service, give it its own week and a new length, press Save once: the row now shows both. Open it again and Cancel after changing something: nothing changed. As a member, each service has "See", which opens the card with nothing to change.

Implementation walkthrough

One step, as the spec planned (light), then four review rounds. Frank asked for it on 2026-10-25, right after the one-card fix on People merged, when he read Services' second row ("Runs whenever the

business is open", with its own Save) as a second card.

frontend/components/settings: the service's card

`services-screen.tsx` draws one box per service. The closed row says its length, Live or Hidden, and when it runs (`ownHoursLine` : "Runs whenever the business is open", or "Runs on its own week" with its days and any one-off dates); `hoursCardOf` and the second row are gone. Change opens `service-form.tsx` , which now holds the hours part beside its own fields. Its one Save takes the hours part's handle once (`hoursNow`), runs `check()` first, then saves the service, who does it (only when the ticks changed) and the hours (only when edited), each through its existing route. A refusal keeps the card open; `unsaved` records what was not saved ("ticks", "ticks-and-hours" when edited hours never went either (F-513), or "hours"), and Cancel tells the page what was saved and what was not. A new service gets no hours part; a member gets "See", the whole card in a disabled fieldset, no Save and Close.

frontend/components/settings: one hours part for both pages

`person-hours-part.tsx` became `own-hours-part.tsx` : the caller brings its title, words and save call, so People and Services share one part and never drift. Its handle gained `edited()` for F-513. `own-hours-card.tsx` and `service-hours-card.tsx` had no reader left and are deleted, with the words only they read (`hoursWords.notSaved` , `bookingsOutside` , `servicesWords.saveWhenItRuns` , `savedWhenItRuns` , `outsideItsHours`) and `useOwnHoursWatch` .

frontend/components/settings: what the page says after a save

The page holds two lists, each under its own title: the bookings a new length no longer fits (`heldTitle`) and the ones new hours leave outside (`heldOutsideTitle`); the card merges every hours answer by booking id while it is open. One notice says what was saved and how many bookings each list holds.

frontend: no other card while one saves (People and Services)

The reviews found a family of races: a card left or another opened while a Save still ran could close the wrong card (F-512), hide a refusal (F-514) or, on People, hand the rest of a Save to the new card's parts (F-511). `frontend/lib/use-report-saving.ts` lets each card (`ChangeResourceForm` , `AddResourceForm` , `ServiceForm`) tell its page while its Save runs; the page disables every other Change and Add, and the card's own Cancel waits too, so no card opens or closes before the answer is shown. A Save that ends after another card opened still closes only its own card (`close(focusId, card)` , F-512). The focus effect on both pages waits until nothing is saving, so the keyboard returns to the row's button once it is enabled again (F-515).

frontend/lib/dashboard-words

`services-words.ts` gains "See", "Close", the read-only title, the outside-its-hours notice and title, and the two "saved, but … was not" notices, in English and Spanish.

not changed

No backend, shared or API change: every save keeps its own route, checks and refusals. Fields stay editable while a card saves; an edit typed during the round trip is not part of that Save.

Findings

one-card-per-service-on-services/F-511 [P3] closed - Opening another card while a Save is still running hands the rest of the first card's chain to the new card's parts

File: frontend/components/settings/people-screen.tsx:147-161, 309-319; frontend/components/settings/change-resource-form.tsx:185-190 **Found:** 2026-10-25 by /audit independent current (round 3, scope: a2efedb..7cb575c; lenses: quality, security, performance, tests) **Why it matters:** `beforeSave` reads `skillsPart.current`, `textsPart.current` and `hoursPart.current` after each `await`, and every other row's Change (and Add) stays clickable while the open card shows "Saving...". If the owner opens Ana's card while Mei's Save is still between two parts, Mei's card unmounts, the refs now point at Ana's parts, and Mei's chain carries on with them: Ana's untouched texts and hours parts answer "nothing changed", so Mei's own texts or hours edits are never sent; then `saveResource` for Mei succeeds and `saved` closes Ana's freshly opened card with "Mei saved". If instead Mei's card is refused after an hours answer arrives late, the parked list is later said under Ana's name by `reportUnfinished`. The ref reading predates this fix (base a2efedb's `beforeSave` read skills and texts the same way); this fix adds the hours part to the chain, where its edits used to have their own Save, and adds the `reportUnfinished` path. Needs a click during a network round-trip, so narrow. Reasoned from the code path; not run in a browser (no login available to this reviewer). **Suggested fix:** Take the three handles once at the start of `beforeSave` (`const parts = { skills: skillsPart.current, ... }`) so the chain stays with its own card, or disable other rows' Change and Add while the open card is submitting. **Resolution:** Fixed 2026-10-25 (Frank: "fix now"), on fix/one-card-per-service-on-services with F-514, the same race: while a card saves, People disables every other card's Change and Add (useReportSaving in frontend/lib/use-report-saving.ts; ChangeResourceForm and AddResourceForm report their isSubmitting), so no other card opens before the Save's answer is shown. Checked on http://localhost:3400 with every save request held back 2 seconds in the page: on Services, Save on Body Wrap, and Change on Chemical Peel was disabled until the answer came, then enabled again; on People, Save on Mei, and Change on Ana was disabled the same way, then "Saved. Mei is on." Reopened 2026-10-25 by /audit independent current (f396e32): re-read people-screen.tsx:143-161, 184-187, 337, change-resource-form.tsx:69, 180 and use-report-saving.ts:6-11. Change and Add are disabled while the card reports `isSubmitting`, but the card's own Cancel stays enabled during the Save, and `useReportSaving`'s unmount cleanup reports `false` as soon as the card closes. So Cancel mid-Save on Mei re-enables every Change; Change on Ana then opens her card while Mei's `beforeSave` is still between awaits, and the chain reads `textsPart.current` and `hoursPart.current` from Ana's card, the original defect, after which `saved` closes Ana's card (People's `close` is unconditional) with "Saved. Mei is on." Even with no second card, Cancel mid-Save leaves the refs null, so Mei's later texts and hours

edits are skipped without a word while `saveResource` still runs and says "Saved". Still narrow (Cancel, then a click, during a round trip); reasoned from the code path, not run in a browser. Suggested: disable Cancel while `isSubmitting` in ChangeResourceForm (and AddResourceForm), or take the three handles once at the start of `beforeSave` and close only the card that saved, as Services does. Fixed again 2026-10-25: the saving card's own Cancel is disabled while it saves too (change-resource-form.tsx, add-resource-form.tsx), so no way out of a card opens another before the Save's answer. Checked on http://localhost:3400 with every save request held back 2 seconds in the page: Services, Save on Body Wrap: its Cancel and Chemical Peel's Change were disabled until the answer; after it, focus was on "Change Body Wrap", and Cancel on Chemical Peel's card returned focus to "Change Chemical Peel". People, Save on Mei: her Cancel and Ana's Change were disabled; after it, focus was on "Change Mei". Closed 2026-10-25 by /audit independent current (a2aa379): re-read people-screen.tsx:65, 138-153, 157-162, 239-240, 263-264, 338-344, change-resource-form.tsx:69-72, 176-187, add-resource-form.tsx:49, 105-113 and use-report-saving.ts:6-12. `beforeSave` runs inside the form's `handleSubmit`, so the card reports `isSubmitting` for the whole chain; while it does, every other Change and Add is disabled and the card's own Cancel (or Close) is disabled too, and `openForm` and `cancelled` are reached only through those buttons. The open card therefore cannot unmount or change mid-chain, so `skillsPart`, `textsPart` and `hoursPart` stay this card's parts from the first await to `saved`. `saving` cannot stick: a refusal leaves the card mounted with `isSubmitting` false (reported by the first effect), and a success unmounts it (reported by the cleanup). No new defect from the repair. Reasoned from the code; not run in a browser (no dashboard login for this reviewer).

one-card-per-service-on-services/F-512 [P3] closed - A Services Save still running closes whichever card the owner opened meanwhile

File: frontend/components/settings/services-screen.tsx:71-78; frontend/components/settings/service-form.tsx:182-231 **Found:** 2026-10-25 by /audit independent current (scope: d46b7bf..67f8845; lenses: quality, security, performance, tests) **Why it matters:** `save` now takes the hours handle once (`hoursNow`), so the parts no longer mix, which is what the spec asked. But every other row's Change stays clickable while the open card says "Saving...", and when the first card's chain finishes, `saved` calls `close(changeButtonId(service.id))`, which sets `open` to null unconditionally: the card the owner opened meanwhile closes, anything typed in it is dropped, and the notice speaks of the first service. The unconditional close predates this fix (base `saved` did the same); this fix lengthens the chain by the hours call, widening the window. Needs a click during a network round-trip, so narrow. Reasoned from the code path; not run in a browser (no login available to this reviewer). **Suggested fix:** Close only when the card still open is the one that saved (`setOpen((now) => (now === service.id ? null : now))`, and the same for "new"), or disable other rows' Change and Add while a card is submitting. **Resolution:** Fixed 2026-10-25 (Frank: "fix both"): services-screen.tsx closes only the card that saved (`close(focusId, card)` with `setOpen((now) => (now === card ? null : now))`; each card passes its own id, or "new"). Checked on http://localhost:3400/settings/services with the services PUT held back 2 seconds in the page: Save on Body Wrap, then Change on Chemical Peel; when Body Wrap's Save ended, Chemical Peel's card stayed open and the notice said Body Wrap saved. Closed 2026-10-25 by /audit independent

current (fc4ae14): re-read services-screen.tsx:66-70 and 72-112. `close(focusId, card)` now leaves another open card alone (`setOpen((now) => (card === undefined || now === card ? null : now))`), each card passes its own id or "new", and the `returnFocus` left set when nothing closes is harmless: the focus effect runs only once `open` is null, and every path there goes through `close` , which overwrites it. No new defect from the repair; the refusal half of the same race is F-514.

one-card-per-service-on-services/F-513 [P3] closed - Cancel after a refused "who does it" says only the ticks were not saved, though the card's hours edits were never sent either

File: frontend/components/settings/service-form.tsx:208-220, 249-252; frontend/components/settings/services-screen.tsx:94-99 **Found:** 2026-10-25 by /audit independent current (scope: d46b7bf..67f8845; lenses: quality, security, performance, tests) **Why it matters:** The one Save runs service, ticks, then hours. When the ticks are refused it returns before the hours call and sets `unsaved = "ticks"` . If the owner then presses Cancel, `cancel` reports "Saved, but who does X was not." while any edits in "When it runs" are dropped without a word. The spec says Cancel "says what was saved"; here it implies everything but the ticks was saved. Narrow (a ticks refusal plus an hours edit in the same Save) and the row then shows the old hours line, so the owner can see it. **Suggested fix:** When the ticks are refused and the hours part is edited, say both were not saved (one word for "who does it and when it runs"), or record the hours as unsaved too. **Resolution:** Fixed 2026-10-25 (Frank: "fix both"): when who does it is refused and the hours part is edited (a new `edited()` on the hours part's handle), the card records "ticks-and-hours", and Cancel says "Saved, but who does Body Wrap and when it runs were not." (Spanish: "Guardado, pero no se guardó quién hace Body Wrap ni cuándo se ofrece."). Checked on http://localhost:3400/settings/services with the resources PUT answering a refusal in the page: Priya unticked and its own week picked, Save showed the refusal in the card, Cancel gave that notice, and Body Wrap still ran whenever the business is open. Closed 2026-10-25 by /audit independent current (fc4ae14): re-read service-form.tsx:182-232 and 250-253 and own-hours-part.tsx:27-31, 55, 62-63. `hoursNow` is taken before the first await, its `edited()` reads that render's `isDirty` , a ticks refusal records "ticks-and-hours" only when the hours part is edited, and Cancel's `outcome()` carries it to `savedButTicksAndHoursNot` , present in English and Spanish in services-words.ts. No new defect from the repair.

one-card-per-service-on-services/F-514 [P3] closed - A Services Save refused after the owner opened another card is never said, and what it already saved never reaches the list

File: frontend/components/settings/services-screen.tsx:160, 199; frontend/components/settings/service-form.tsx:208-229, 250-253 **Found:** 2026-10-25 by /audit independent current (scope: d46b7bf..fc4ae14; lenses: quality, security, performance, tests) **Why it matters:** A service's card is drawn only while `open === service.id` , and every other row's Change (and Add) stays clickable while it says "Saving...". If the owner opens another card mid-Save, the first card unmounts but its chain runs on. When that chain then ends on a refusal (who does it, or its hours, after the service itself saved), the refusal goes to the unmounted card's notice or hours part and is never seen, its

`unsaved` is lost, and Cancel, the only path that hands `savedService` to the list, can no longer be pressed: the row keeps the old name, length or Live state until a reload, with no word that who does it or when it runs was not saved. The ticks half predates this fix (base's chain had the same shape); this fix adds the hours call to the chain, widening it. Narrow (a click during a round trip, then a refusal). Reasoned from the code path; not run in a browser (no login available to this reviewer). **Suggested fix:** Disable other rows' Change and Add while a card is submitting, which closes this and F-512's race in one place, or have a card that is no longer the open one report a refusal through the parent (`onSaved(savedService.current, outcome())`). **Resolution:** Fixed 2026-10-25 (Frank: "fix now"): while a card saves, Services disables every other card's Change and Add (ServiceForm reports its isSubmitting through useReportSaving), so a refusal always lands in its own open card. Fixes People's F-511 the same way. Checked on http://localhost:3400 with every save request held back 2 seconds in the page: on Services, Save on Body Wrap, and Change on Chemical Peel was disabled until the answer came, then enabled again; on People, Save on Mei, and Change on Ana was disabled the same way, then "Saved. Mei is on." Reopened 2026-10-25 by /audit independent current (f396e32): re-read services-screen.tsx:56-57, 68-72, 151, 203, service-form.tsx:176, 186-236, 254-257, 540 and use-report-saving.ts:6-11. Opening another card mid-Save is now blocked, but Cancel stays enabled while the card says "Saving...", and the unmount cleanup then re-enables every Change. Cancel pressed after the service itself saved but before who does it or its hours answer calls `onSaved(savedService.current, outcome())` with `unsaved` still null, so the page says "Saved. Customers can book Body Wrap now." and closes the card; the chain runs on in the unmounted card, and a refusal of the ticks or the hours lands there and is never said, while the notice claims the save went through. The list half of the original defect is repaired (Cancel hands the saved service to the list); the "never said" half is still reachable. Narrow; reasoned from the code path, not run in a browser. Suggested: disable Cancel while `isSubmitting`, which also closes F-511's Cancel path. Fixed again 2026-10-25: the saving card's own Cancel is disabled while it saves too (service-form.tsx), so a refusal always lands in its own open card. Checked on http://localhost:3400 with every save request held back 2 seconds in the page: Services, Save on Body Wrap: its Cancel and Chemical Peel's Change were disabled until the answer; after it, focus was on "Change Body Wrap", and Cancel on Chemical Peel's card returned focus to "Change Chemical Peel". People, Save on Mei: her Cancel and Ana's Change were disabled; after it, focus was on "Change Mei". Closed 2026-10-25 by /audit independent current (a2aa379): re-read services-screen.tsx:56-57, 63-73, 137-146, 149-153, 169-181, 199-210, service-form.tsx:176, 186-236, 254-257, 530-548 and use-report-saving.ts:6-12. While the card's Save runs, its own Cancel (or Close) is disabled (`disabled={form.formState.isSubmitting}`) and every other Change and Add is disabled through `saving`, so the card stays mounted until `save` ends: a refusal of who does it or its hours lands in this open card (its notice, or the hours part's own error), `unsaved` stays set, and the later Cancel hands `savedService` and that word to the list through `onSaved(savedService.current, outcome())`. Neither half of the original defect is reachable now. No new defect from the repair. Reasoned from the code; not run in a browser (no dashboard login for this reviewer).

one-card-per-service-on-services/F-515 [P2] closed - After every Save, the keyboard focus is lost instead of returning to the row's Change button

File: frontend/lib/use-report-saving.ts:6-11; frontend/components/settings/services-screen.tsx:63-72, 203; frontend/components/settings/people-screen.tsx:138-146, 337 **Found:** 2026-10-25 by /audit independent current (scope: d46b7bf..f396e32; lenses: quality, security, performance, tests) **Why it matters:** Both pages promise "where the keyboard goes once a card closes: back to the button that opened it, never the top". A successful Save calls `saved` -> `close`, which sets `open` to null while the page's `saving` is still `true`: `saving` turns false only from the card's own effects, and the card unmounts in that same render, so its cleanup (`onSavingChange(false)`) runs after the render that draws the row. In that commit the row's Change button is drawn with `disabled={saving}` true, the card's unmount cleanup only schedules the next render, and the page's `[open]` effect then calls `.focus()` on a disabled button, which does nothing, and clears `returnFocus`. Focus is left on the document body (the Save button that held it was removed), so a keyboard or screen-reader owner starts again from the top of the page after every Save on Services and People, Add included (the new row's Change). Before this delta the buttons were never disabled and focus returned. Reasoned from React's commit order (passive unmount cleanups, then mount effects; state set in a cleanup applies in a later render); not run in a browser (no dashboard login for this reviewer). To confirm: Save a card, then read `document.activeElement`. **Suggested fix:** Clear `saving` in the same update as the close (`setSaving(false)` beside `setOpen` in `close`), or let the focus effect wait on `saving` too (`[open, saving]`, returning while `saving`), so the button is enabled before it is focused. **Resolution:** Fixed 2026-10-25: the focus effect on People and Services waits until no card is saving (`[open, saving]`, returning while `saving`), so the row's button is enabled before it takes the focus. Checked on http://localhost:3400 with every save request held back 2 seconds in the page: Services, Save on Body Wrap: its Cancel and Chemical Peel's Change were disabled until the answer; after it, focus was on "Change Body Wrap", and Cancel on Chemical Peel's card returned focus to "Change Chemical Peel". People, Save on Mei: her Cancel and Ana's Change were disabled; after it, focus was on "Change Mei". Closed 2026-10-25 by /audit independent current (a2aa379): re-read services-screen.tsx:63-73, 152, 204, people-screen.tsx:138-147, 239, 338 and use-report-saving.ts:6-12. Both effects now return while `saving` and depend on `[open, saving]`. After a Save, `close` sets `open` to null and `returnFocus` while `saving` is still true, so the effect waits; the closed card's unmount cleanup then sets `saving` false, and in that next commit the row's Change (or Add) is drawn enabled before the effect focuses it. Cancel on a card that is not saving finds `saving` already false and focuses at once, and a refusal keeps the card open, so `open` is not null and nothing is focused there. `returnFocus` is cleared only after a focus attempt. No new defect from the repair. Reasoned from React's commit order; not run in a browser (no dashboard login for this reviewer).

Independent review

Status: passed **Target commit:** a2aa3795ae18e0f3c00df3c551de3f8a0a14351e **Base commit:** d46b7bff43c7afad9b94b7e70ecb01d32d31e00f **Base ref:** main **Spec hash:** 8196bb5bdea8182017c411141783ca5b6f2079649737b20d15518d0b5bee9f4a **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-10-25T16:04:42Z **Workflow:** regular **Check**

required: no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-10-25T16:08:54Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Commands

- `npm run test --workspace=@scheduleads-app/shared` : pass (32 files, 214 tests)
- `npm run test --workspace=backend` : pass (95 files, 1098 tests)
- `npm run test --workspace=@frankdmosquera/booking-component` : pass (18 files, 98 tests)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass (compiled, TypeScript finished)
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass

Evidence

- Freshness: HEAD a2aa379 equals the target, `git merge-base main HEAD` is d46b7bf, the spec's SHA-256 matches, and only `blueprint/context/review.md` differed from the target before this receipt.
- Delta reviewed in full (12 frontend files plus the spec): services-screen.tsx, service-form.tsx, own-hours-part.tsx (renamed from person-hours-part.tsx, a faithful generalisation: the save call and words moved to the caller, plus `edited()`), own-hours-fields.tsx, people-screen.tsx, change-resource-form.tsx, add-resource-form.tsx, use-report-saving.ts, services-words.ts, hours-words.ts, and the deleted own-hours-card.tsx and service-hours-card.tsx (`git grep` finds no reader of them or of the removed words).
- Spec against code: one box per service with its hours line (services-screen.tsx:184-219); one Save runs hours check, service, who does it, hours (service-form.tsx:186-236); Cancel says what was saved (254-257); no hours part for a new service (140, 501); members get See, a disabled fieldset, no Save and Close (175, 280, 531-548); new English and Spanish words present in both dictionaries.
- Security: no backend or shared change in the delta; the read-only card is a display choice, and the save routes keep their own server-side role checks (unchanged, covered by the backend suite).
- Performance: no new requests or loops; the closed row's hours line is a per-row string join.
- F-511, F-514 and F-515 re-examined against the code and closed; F-512 and F-513 stay closed.

Findings

- F-511 closed, F-514 closed, F-515 closed. No new findings.

Remaining risk

- UI paths judged from the code only: this reviewer has no dashboard login, so the cards, the Save order, focus return and the disabled buttons were not exercised in a browser here (the builder's ledger records browser checks with held-back requests).
- The frontend has no test runner, so the one-Save order and the Cancel wording have no automated test; the backend suite covers the routes the Save calls.
- Fields stay editable while a card saves; an edit typed during the round trip is not part of that Save (the usual form behaviour, unchanged by this fix).